Johnson&Johnson
MedTech

Diagnostic Report

Report Title	Finding the Lag: A Plain-Language Look at the Robotic Arm's Response Times
Name	Ossama Barko
Date	3 October 2026
Ticket ID	#2437

Executive Summary

- **Purpose:** This report explains how I looked into slow responses from the robotic arm's control commands, what I found, and what I would do next. I have tried to write it so that engineers and non-engineers can both follow it.
- **Key Findings:** All three commands (move_arm, rotate_joint and adjust_grip) responded close to, or faster than, their expected times. rotate_joint was the slowest at 0.15 seconds, so it was my main focus. A simulated 20% optimization brought it down to about 0.12 seconds and moved move_arm from just over its target to comfortably under it.

Issue Description

- **Problem Statement:** Delays were reported in the robotic arm's control system, and the rotate_joint command was flagged as the main concern (Ticket #2437).
- **Symptoms and Impact:** When a command responds late, the arm reacts a fraction of a second behind the surgeon's input. In surgery, even small lags can make movements feel less precise, force the surgeon to compensate, and, in the worst case, affect patient safety. In my tests the measured times were within or very close to their targets, so the immediate risk looks low, but small delays can add up across many commands and are worth tracking.

Johnson&Johnson
MedTech

- **Client Information:**
  - **Hospital Name:** [To be added from Ticket #2437]
  - **Location:** [To be added from Ticket #2437]
  - **Reported By:** [To be added from Ticket #2437]

Diagnostic Process

- **Initial Observations:** Before any changes, move_arm took 0.102 seconds, rotate_joint took 0.15 seconds and adjust_grip took 0.051 seconds.
- **Commands Tested:** move_arm (moves the arm to a position), rotate_joint (turns a joint, and the command flagged for delay) and adjust_grip (opens or closes the gripper).
- **Hypothesis:** I suspected rotate_joint was slowed by a blocking wait or redundant calculations in its code. In the notebook the delay is a fixed 0.15 second pause, so this hypothesis describes what a real system would most likely contain.
- **Tools and Techniques Used:** Python 3 (the time and numpy libraries), the Control System Diagnostic Notebook run in VS Code, and iterative testing: measure each command, apply a change, then measure again.

Findings and Analysis

Response Time Data:

Command	Expected Response Time	Initial Response Time	Optimized Response Time
move_arm	0.10 seconds	0.102 seconds	~0.082 seconds
rotate_joint	0.18 seconds	0.15 seconds	~0.12 seconds
adjust_grip	0.09 seconds	0.051 seconds	~0.041 seconds

- **Analysis of Findings:** No command was far outside its target. rotate_joint was the slowest in absolute terms (0.15 seconds) but still within its 0.18 second limit. move_arm was 2 milliseconds over its 0.10 second target, which looks like small timing overhead rather than a real fault. After the simulated optimization, every command was at or below its expected time.

Johnson&Johnson
MedTech

Optimizations and Solutions

- **Code modifications:** The notebook's optimized_command function applies a simulated 20% improvement (improvement_factor = 0.2) to each command's measured time. No real command code was changed. In a live system, I would start by removing redundant calculations and blocking waits from rotate_joint.
- **Impact of Optimizations:** The simulation cut rotate_joint by about 0.03 seconds and brought move_arm under its target. Because this gain is simulated, I recommend re-testing after real code changes: repeat each measurement several times and average the results, use time.perf_counter() for finer timing, and run stress tests under heavy load.

Conclusion

- **Summary of Findings:** rotate_joint was the slowest command at 0.15 seconds but stayed within its limit, and move_arm was 2 milliseconds over target. A simulated 20% improvement showed that all three commands can come in under their expected times.
- **Overall Impact:** Faster, more consistent responses mean the arm follows the surgeon's input more closely, which supports precision and reliability during procedures. These figures come from a simulation, so the real-world gain still needs to be confirmed on the actual system.
- **Next Steps:** First, find the real cause of the rotate_joint delay in the live control code. Second, apply one change at a time and re-measure. Third, repeat and average the timings. Finally, stress-test under high load before any clinical use.